

DBSCAN: Aprendizaje Basado en Densidad

2026

Índice

1. Introducción	2
2. Funcionamiento del Algoritmo	2
2.1. Hiperparámetros del Modelo	2
2.2. Clasificación de las Observaciones	2
3. Ventajas y Desventajas	3
4. Casos de Uso Reales	3
5. Implementación en Python	5
5.1. Uso de Sklearn	5
5.2. Elección de Parámetros	5
6. Conclusiones	5
7. Ejemplos	6

1. Introducción

DBSCAN (*Density Based Spatial Clustering of Applications*) es un algoritmo de clusterización altamente reconocido debido a que, a diferencia de métodos como K-means, posee la capacidad de clusterizar correctamente formas de datos complejas.

¿Qué es densidad? ¿Qué concepto nos puede parecer similar?

2. Funcionamiento del Algoritmo

El nombre del algoritmo indica su naturaleza: es un método de clusterización basado en la **densidad**. La densidad se define como el número de observaciones dispuestas en una misma zona.

2.1. Hiperparámetros del Modelo

Dado que el concepto de densidad puede ser subjetivo, el modelo se rige por dos hiperparámetros fundamentales:

1. **Epsilon (ϵ):** La distancia máxima permitida entre puntos para ser considerados vecinos.
2. **MinSamples:** El número mínimo de puntos necesarios para considerar una zona como “densa”.

2.2. Clasificación de las Observaciones

DBSCAN clasifica cada punto en una de tres categorías:

- **Puntos Core:** Puntos que cumplen estrictamente con las condiciones de densidad fijadas.
- **Puntos Alcanzables (*Border points*):** Puntos que no cumplen la densidad mínima pero están dentro del radio de un punto core. (Nota.- En terminología habitual se denominan density-reachable y border points)
- **Ruido:** Observaciones que no cumplen condiciones de densidad ni tienen puntos core cerca.

3. Ventajas y Desventajas

A pesar de su sencillez operativa, DBSCAN ofrece capacidades bastante interesantes:

Ventajas	Desventajas
Detecta formas geométricas complejas.	Sensible al orden de los datos en puntos frontera.
Gestión correcta de <i>outliers</i> (ruido).	Complejo de parametrizar en regiones con densidades variables.
Generación automática del número de clústeres.	
Parámetros intuitivos.	

Cuadro 1: Análisis comparativo de DBSCAN.

4. Casos de Uso Reales

El algoritmo es preferido sobre K-Means en situaciones donde el ruido es factor determinante o las formas no son esféricas:

- **Detección de fraude bancario:** Identifica comportamientos habituales como clústeres y operaciones sospechosas como ruido.
DBSCAN:

- Comportamientos habituales → clusters
- Operaciones sospechosas → ruido
- Ideal porque identifica anomalías automáticamente.
- No necesita saber cuántos tipos de fraude hay.

K-Means

- Obliga a que todos los puntos pertenezcan a un cluster.
- Las operaciones fraudulentas se mezclarían con operaciones normales Resultado: no detecta bien el fraude

- **Análisis geográfico (Smart Cities):** Detecta zonas de alta actividad (*hotspots*) con formas irregulares siguiendo calles o barrios. DBSCAN:
 - Agrupa puntos GPS (taxis, móviles, etc.).
 - Permite detectar:
 - Zonas con alta actividad
 - Rutas frecuentes
 - Puede encontrar clusters con formas irregulares (calles, barrios).

Muy usado en smart cities.

K-Means

- Genera clusters circulares.
 - Divide artificialmente zonas reales.
 - Ejemplo: un barrio alargado → lo corta en varios clusters. Resultado: pierde sentido geográfico
- **Biología y Medicina:** Agrupación de células o genes donde los datos suelen incluir ruido y formas complejas. Ayuda a detectar patrones en datos médicos.

Es útil cuando los datos no siguen formas simples DBSCAN:

- Los datos (células, genes) suelen tener formas complejas e incluir ruido
- detecta grupos naturales
- ignora datos atípicos

K-Means

- Obliga a definir el número de grupos (muchas veces desconocido).
- Sensible a outliers:
 - ◊ puede distorsionar los clusters
 - ◊ Resultado: agrupaciones poco fiables científicamente
- **Segmentación de clientes:** Permite detectar clientes atípicos de alto valor o riesgo que K-Means diluiría en grupos generales.

DBSCAN:

- grupos reales de comportamiento
- clientes atípicos (muy importante en negocio)
- No obliga a un número fijo de segmentos.

K-Means

- Necesitas decidir “k” (n^0 de clientes tipo).
- Todos los clientes son asignados a un grupo: incluso los raros
- Resultado: pierdes clientes especiales (alto valor o riesgo)

5. Implementación en Python

5.1. Uso de Sklearn

La implementación estándar se realiza a través del módulo `cluster`:

```
from sklearn.cluster import DBSCAN

# Ajuste y prediccion en un solo paso
clusters = DBSCAN(eps=3, min_samples=5).fit_predict(data
)
```

5.2. Elección de Parámetros

Para optimizar ϵ , se recomienda utilizar el gráfico de distancias mediante el modelo *Nearest Neighbors* de Sklearn. Este gráfico permite visualizar la distancia al vecino más cercano y encontrar el “punto de codo” óptimo.

6. Conclusiones

- **Usar DBSCAN:** Cuando existen anomalías, formas complejas o se desconoce el número de clústeres.
- **Usar K-Means:** Para clústeres esféricos simples y cuando se conoce el número de grupos en datos limpios.

El algoritmo DBSCAN es especialmente adecuado en contextos donde:

- existen datos con Ruido
- no se conoce el número de grupos
- las estructuras no son lineales o esféricas

Y no es recomendable cuando:

- la densidad es muy variable
- alta dimensionalidad

A diferencia de K-Means, DBSCAN:

- no necesita definir k
- identifica ruido
- detecta formas complejas

7. Ejemplos

- Ejemplo DBSCAN - SIG
- Ejemplo DBSCAN - Outliers